Decentro SDE Intern Assignment — Expense Sharing Service

Overview

Imagine a group of friends goes on a trip together.

Throughout the trip, different people pay for meals, transport, hotel bookings, and activities. At the end of the trip, the group wants to understand:

- How much each person spent
- How much each person owes
- Who should pay whom

Your task is to build a backend service that helps a group track shared expenses and calculate balances between members.

This assignment is designed to evaluate:

- Database design skills
- Problem-solving and logical reasoning
- Backend engineering fundamentals
- Code quality
- Creativity and product thinking

Tech Stack

Backend

Preferred:

- Python

Frameworks:

- FastAPI
- Flask
- Django

Database

You may use:

- PostgreSQL
- MySQL
- SQLite

Objective

We want to evaluate:

- Database design
- Backend/API design
- Business logic implementation
- Code quality
- Problem-solving ability
- Creativity

Problem Statement

Build a backend service where users can:

- Create users
- Create groups
- Add users to groups
- Record shared expenses
- View expense history
- View balances between users

Example Scenario

Users:

- Mohit

- Rahul
- Ankit

Group:

Goa Trip

Expense:

```
None
Dinner = ₹1200


Paid By:

Mohit


Participants:

Mohit

Rahul

Ankit
```

Since there are 3 participants:

```
None
₹1200 ÷ 3 = ₹400 per person
```

Mohit has already paid ₹1200.

Therefore:

```
None
Rahul owes Mohit ₹400

Ankit owes Mohit ₹400
```

Expected Balance View:

```
Rahul → Mohit : ₹400


Ankit → Mohit : ₹400
```

Functional Requirements

Part 1 — User Management

Create User

Create a new user.

Example Request

```json
{
  "name": "Mohit"
}
```

Example Response

```json
{
  "id": 1,
  "name": "Mohit"
}
```

List Users

Return all users in the system.

Part 2 — Group Management

Create Group

Example Request

```JSON
{
  "name": "Goa Trip"
}
```

Example Response

```JSON
{
  "id": 1,
  "name": "Goa Trip"
}
```

Add User To Group

Example Request

```JSON
{
  "group_id": 1,
```

```
  "user_id": 2
}
```

List Group Members

Return all members belonging to a group.

Part 3 — Expense Management

Add Expense

Example Request

```JSON
{
  "group_id": 1,
  "paid_by": 1,
  "amount": 1200,
  "description": "Dinner"
}
```

Rules

- Amount must be greater than 0.
- Expense must belong to a valid group.
- Expense should be split equally among all members of the group.
- The payer should not owe themselves.

List Expenses

Return all expenses for a particular group.

Example Response

```JSON
[
  {
    "description": "Dinner",
    "amount": 1200,
    "paid_by": "Mohit"
  }
]
```

Part 4 — Balance Calculation

The system should calculate balances between users.

Example:

Users:

- Mohit
- Rahul
- Ankit

Expense:

```None
Dinner = ₹1200

Paid by Mohit
```

Expected Result:

```
Rahul owes Mohit ₹400


Ankit owes Mohit ₹400
```

Provide an API that returns balances in a structured format.

Example Response

```json
{
  "Rahul": {
    "owes": {
      "Mohit": 400
    }
  },
  "Ankit": {
    "owes": {
      "Mohit": 400
    }
  }
}
```

API Requirements

You are free to design your APIs.

A possible set of APIs could be:

```
None
POST /users

GET  /users


POST /groups

POST /groups/{group_id}/members

GET  /groups/{group_id}/members


POST /expenses

GET  /groups/{group_id}/expenses


GET  /balances
```

You may choose a different API design if you can justify it.

Database Design

Design your own database schema.

You are free to create any tables that you think are necessary.

A possible design may include:

users

- id
- name

groups

- id
- name

group_members

- id
- group_id
- user_id

expenses

- id
- group_id
- paid_by
- amount
- description
- created_at

You may modify this design if you believe a better design exists.

Product Enhancement (Mandatory)

Implement one additional feature of your choice.

There are intentionally no constraints.

The goal is to understand how you think about products, users, and engineering tradeoffs.

Examples:

- Search functionality
- Expense categories
- Monthly summaries
- Spending analytics
- Export functionality
- Smart settlement suggestions
- Any other feature you believe improves the product

Please Explain

- Why you chose the feature

- What problem it solves
- Design decisions made
- Tradeoffs considered

Optional Bonus Features

These are NOT mandatory.

Balance Settlement

Allow users to settle balances.

Example:

```
None
Rahul pays Mohit ₹400
```

Updated Balance:

```
None
Rahul owes Mohit ₹0
```

Unequal Expense Split

Support custom splits.

Example:

```
JSON
{
  "amount": 1000,
  "splits": [
    {
```

```
      "user_id": 1,
      "amount": 500
    },
    {
      "user_id": 2,
      "amount": 300
    },
    {
      "user_id": 3,
      "amount": 200
    }
  ]
}
```

Unit Tests

Add tests for:

- Expense creation
- Balance calculation
- Validation logic

Docker Support

Provide Docker setup.

Submission Requirements

Please submit:

- GitHub Repository
- README.md
- Setup Instructions
- API Documentation
- Database Schema Explanation

Optional:

- Postman Collection
- Demo Video

README Should Include

- Project setup steps
- Architecture overview
- Database design explanation
- Assumptions made
- Design decisions
- Improvements you would make with more time

Evaluation Criteria

Area	Weight
Database Design	30%
Problem Solving & Logic	30%

Backend Implementation	20%
Code Quality	10%
Product Enhancement	10%

Time Expectation

Expected completion time:

10–12 hours

You are NOT expected to build a production-grade application.

We care more about:

- Correctness
- Clean code
- Logical thinking
- Database design
- Engineering decisions

Submission Deadline

(To be shared separately)

Final Notes

Focus on building a clean and functional backend service.

Make reasonable assumptions wherever necessary and document them clearly.

We are more interested in your thought process, design decisions, and problem-solving approach than a perfect implementation.

Good luck!